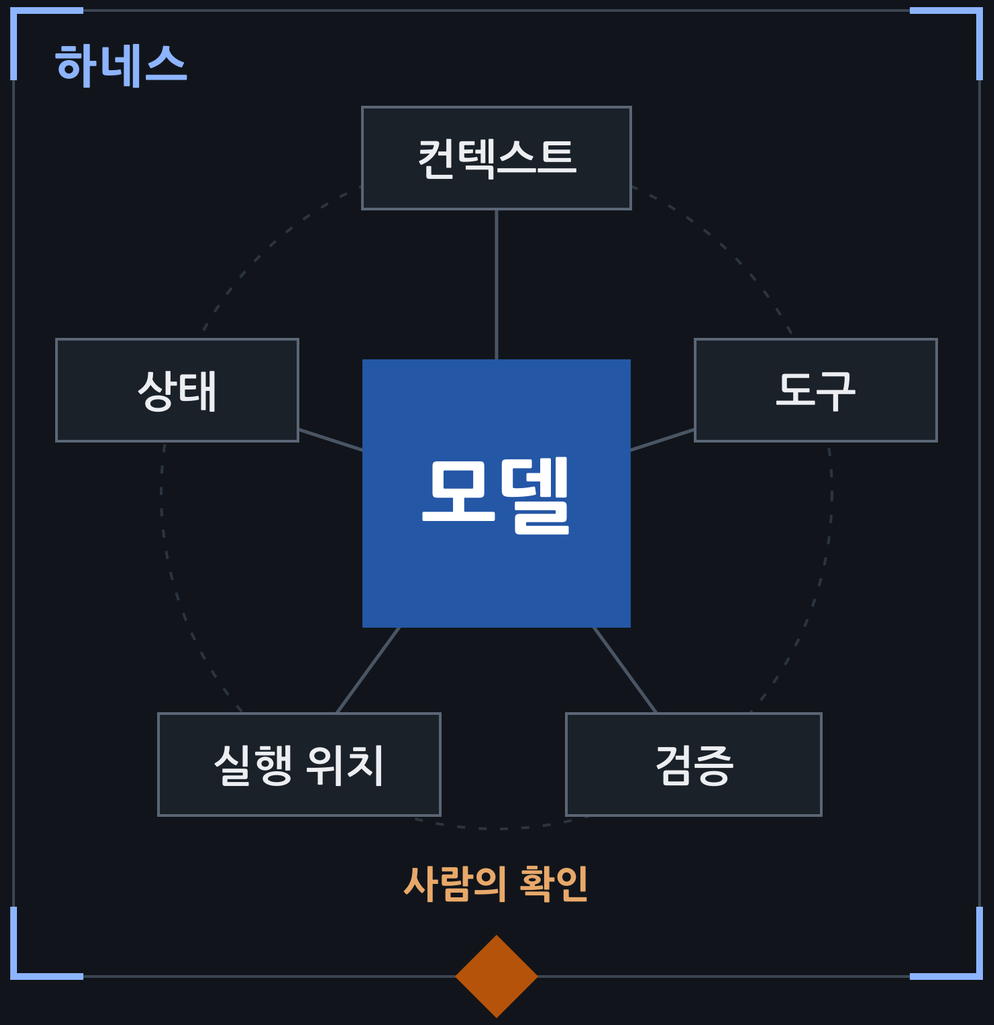
워크숍

하네스 엔지니어링

AI에게 일을 맡기는 구조를 설계하는 법

용어 26개와 사례 5개로 읽는 작업의 틀

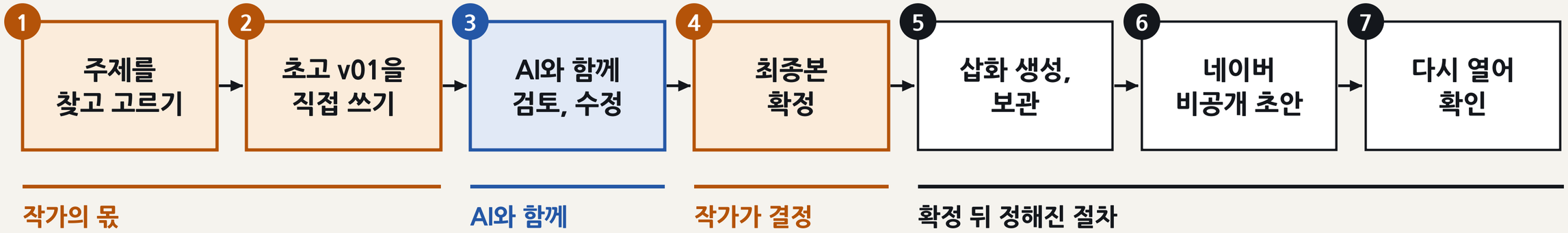


시작

AI교양 글 한 편, 처음부터 끝까지

주제와 초고는 작가가, 다듬기는 AI와 함께, 확정은 다시 작가가. 그 뒤로 삽화, 비공개 초안, 재확인.

v01은 작가가 직접 쓴 원고 (AI 초안 아님)



공개 발행은 이 흐름 밖

단계를 건너뛰지 않게 지켜 주는 건 누구일까, 무엇일까?

설명용 정리 (실제 화면과 계정 정보는 싣지 않음). 낯선 용어는 뒤에서 하나씩.

오늘의 길

질문 다섯 개, 사례 다섯 개

01

AI는 무엇을 하고, 누가 실행할까?

모델, 에이전트, 런타임, 하네스

사례 클식이

02

일하는 방법을 어디에 담을까?

오케스트레이션, 도구, 스킬, 플러그인, 커넥터, MCP

사례 OMH, Grokbot

03

AI는 무엇을 알고 있을까?

프롬프트, 컨텍스트, 토큰, 컨텍스트 창, 메모리, LLM wiki

사례 AI교양

04

누가 이어서 일할까?

세션, 워커, 서브에이전트

사례 CEO Agent

05

끝났다는 말을 어떻게 믿을까?

워크플로우, 자동화, 완료 보고, 검증, 로컬 실행, 클라우드 실행, 리모트 컨트롤

다섯 사례 종합

개념 다음엔 바로 사례. 사례에 먼저 나오는 낯선 용어는 이름만 보고, 설명은 해당 슬라이드에서.

1장

AI는 무엇을 하고, 누가 실행할까?

01

모델의 판단과 실제 실행, 그 사이를 잇는 네 개념. 클식이 구조에서 찾아볼 역할과 경계

모델 — 판단과 생성

에이전트 — 반복과 종료

런타임 — 실제 실행

하네스 — 작업 틀 전체

사례 클식이 — 구조 안에서 찾기

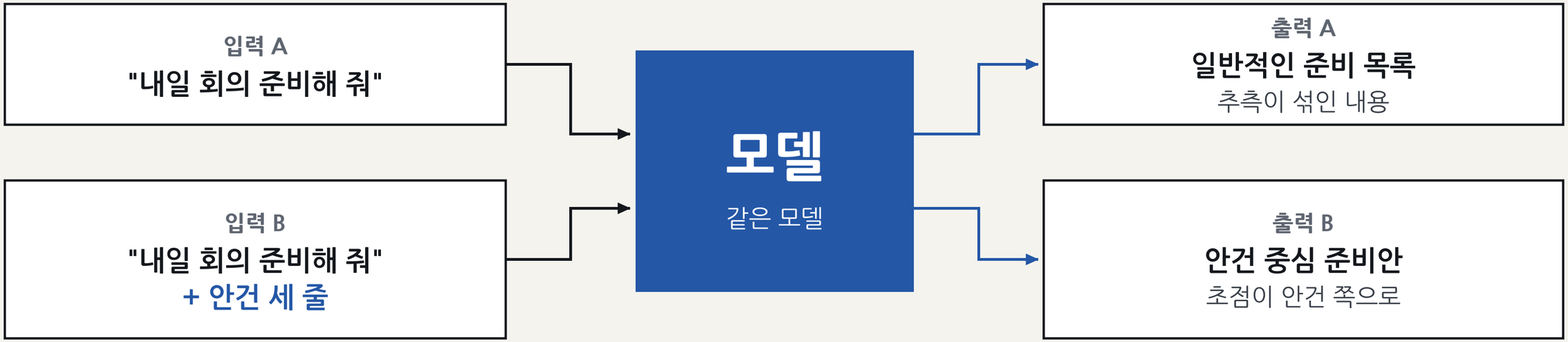
모델

장면

"내일 회의 준비해 줘." 안건도, 참석자도 없이.

그럴듯한 준비안이 나올까?

입력된 텍스트와 맥락을 받아 다음 출력을 만드는 학습된 계산 장치. 그 출력은 답변이거나 도구 호출 요청.

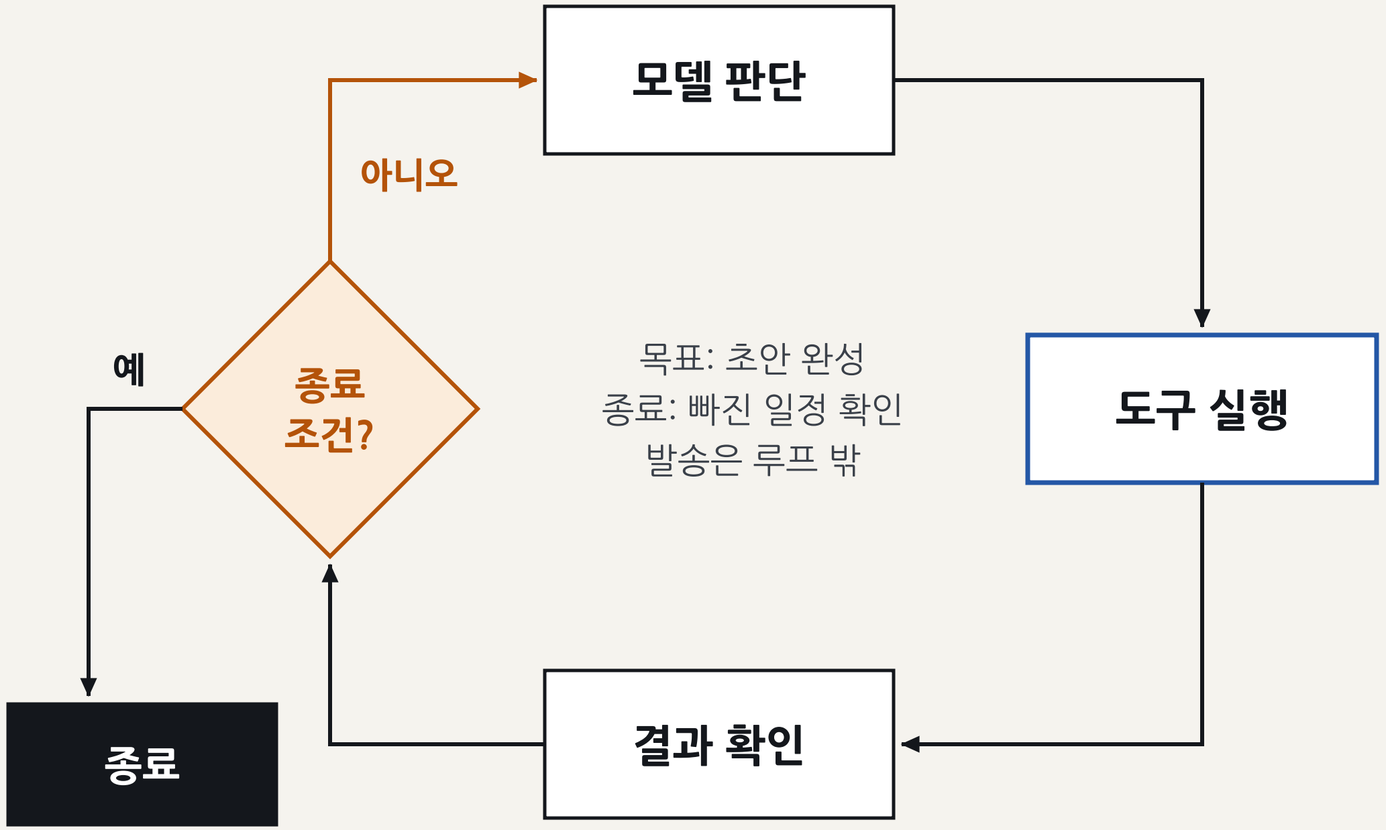


출력이 도구 호출 요청일 때 `read_file(path=tickets.csv)` 요청까지만. 파일은 여기서 열리지 않음

헷갈리기 쉬운 점 모델은 출력을 만드는 쪽. 그 출력이 도구 실행, 결과 확인으로 이어지며 여러 단계를 도는 구성이 에이전트

에이전트

목표를 받아 모델의 판단, 도구 실행, 결과 확인을 되풀이하다가 종료 조건에서 멈추는 실행 구성



장면

일정 안내 메일 초안 맡기기. 빠진 일정은 되물어 채우고, 발송은 빼고.

끝나는 조건이 있는 반복이면 에이전트일까?

헷갈리기 쉬운 점 기준은 응답 횟수가 아니라, 결과를 보고 다음 행동이나 종료를 고르는 구조인지

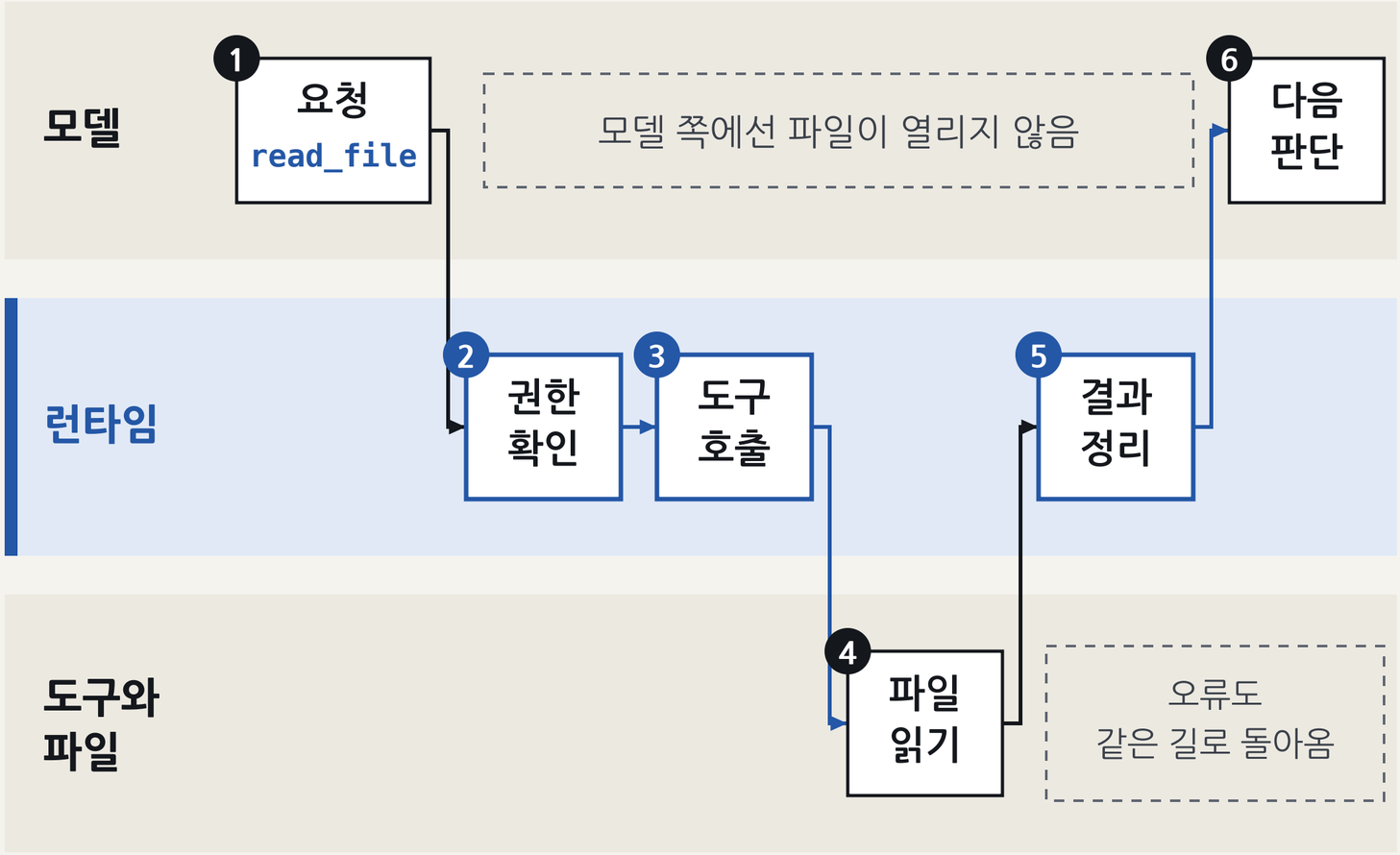
런타임

모델의 도구 호출 요청을 받아 권한을 확인하고 도구를 실행한 뒤, 결과를 다시 모델 입력으로 돌려주는 실행 계층

장면

"지난주 회의 메모 파일 열어 줘."

실제로 파일을 여는 쪽은 누구일까?



헷갈리기 쉬운 점 Python, Node.js 같은 언어 실행 환경과는 다른 말. 하네스보다 좁은, 실행을 잇는 층

하네스

장면

체크리스트 파일을 폴더에 넣어 두고 검토 시작.

파일이 폴더에 있으면 검토 틀이 갖춰진 걸까?

모델이 안정적으로 일하도록 컨텍스트, 도구와 실행 위치, 상태 저장, 검증 절차를 묶은 작업 틀 전체 (이 워크숍의 정의, 제품마다 범위는 다름)

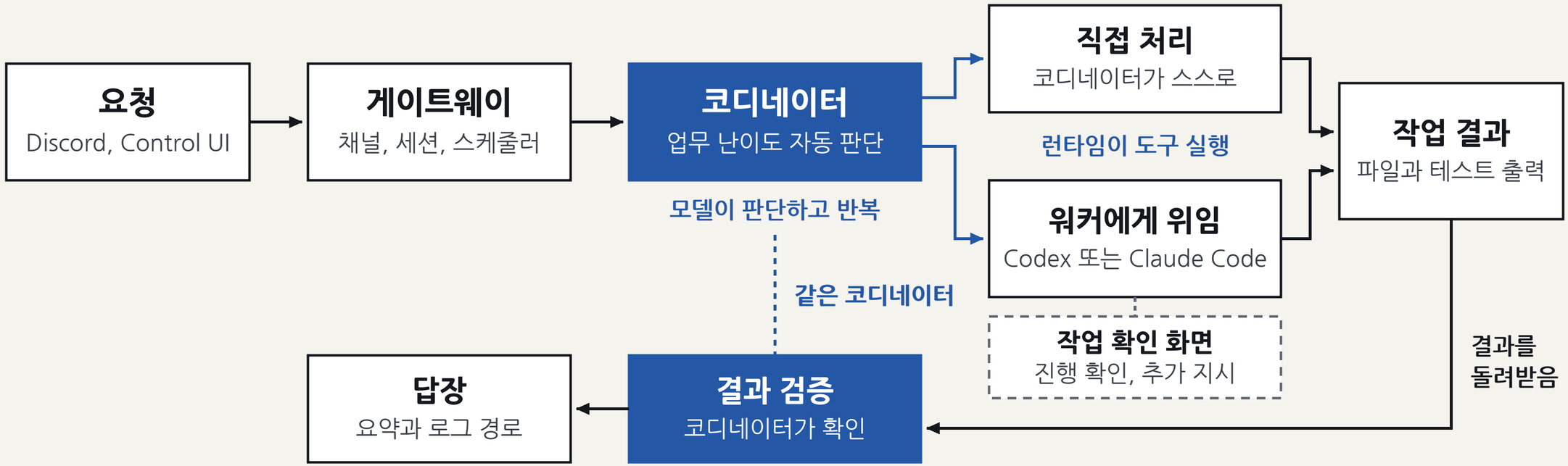


헷갈리기 쉬운 점 하네스는 틀, 오케스트레이션은 그 안의 순서와 역할 배정, 런타임은 실행을 잇는 층

질문 디스코드로 온 "보고서 검증해 줘". 누가 받고, 누가 맡기고, 누가 결과를 확인할까?

요청을 받고, 맡기고, 돌아온 결과를 확인하는 구조

하네스: 이 구조 전체



운영 문서 기준의 가상 예시. 두 파란 상자는 같은 코디네이터의 단계. 결과는 테스트와 파일로 확인

2장

일하는 방법을 어디에 담을까?

02

순서와 역할, 실제 기능, 다시 쓰는 절차. 설치와 연결까지 구분하고 OMH와 Grokbot으로 비교



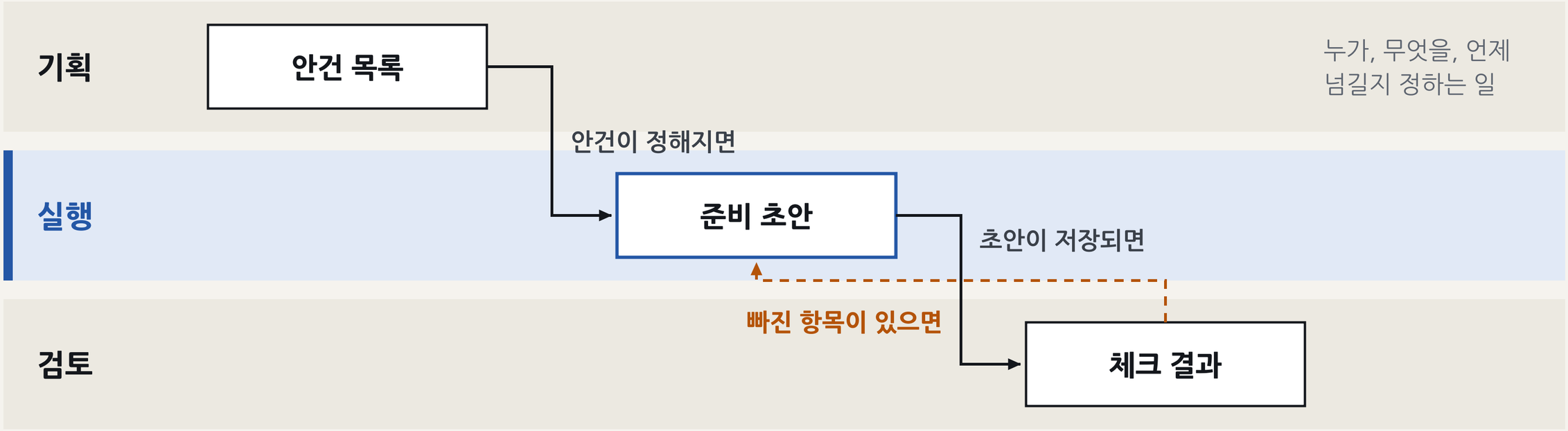
오케스트레이션

장면

회의 준비에 필요한 안건 초안, 확인, 최종 정리. 맡을 사람은 아직 미정.

담당이 없으면 같은 일을 두 번 하게 될까?

여러 단계나 여러 작업자에게 순서, 입출력, 역할, 다음 단계로 넘어가는 조건을 배정하고 조율하는 일



헷갈리기 쉬운 점 역할 분담이 곧 독립 검토는 아님. 에이전트 하나 안에서 단계를 배정하는 것도 오케스트레이션

도구

장면

폴더에서 최근 회의 파일 목록 뽑기.

목록을 가져오는 기능과 찾는 순서 메모, 무엇이 실행될까?

모델이 요청하고 런타임이 실행하는, 이름과 입력 형식과 결과 형식이 정해진 호출 가능한 기능

`list_files` 도구

설명	폴더의 파일 목록 반환
입력	`folder, sort`
결과	파일 이름 목록, 또는 오류

```
list_files(folder=meetings, sort=date)
```

찾는 순서 메모

1. meetings 폴더 열기
2. 날짜순 정렬
3. 최근 파일부터 적기

읽으면 이해되지만 스스로 움직이지는 않음

✓ **실행되는 기능. 요청은 모델이, 실행은 런타임이**

✕ **절차일 뿐. 실행 수단이 있어야 결과가 나옴**

헷갈리기 쉬운 점 "날짜를 ISO 형식으로 바꾼다"는 절차, 그 변환을 실행하는 호출이 도구. MCP는 도구를 잇는 규약

스킬

특정 작업의 절차, 입력과 출력 규칙, 필요한 자원을 묶어 다시 쓸 수 있게 만든 지침 묶음



장면

주간 고객 문의 요약, 매주 같은 순서로.

그 순서를 묶어 둔 것을 무엇이라 부를까?

입력 **새 주간 CSV와 이 묶음** → 출력 **같은 형식의 요약**

헷갈리기 쉬운 점 스킬은 도구가 아니고 스스로 실행되지도 않음. 플러그인은 스킬 등을 배포하는 포장

사례 2 / 5 OMH

질문 OpenCode만으로도 되는 일, OMH로 하면 무엇이 달라질까?

빈칸만 채우랬는데, 번호까지 바뀌었다면?

요청 **표의 금액 빈칸만 0으로 채워 줘. 번호와 상품은 그대로** 같은 모델, 추가 학습 없음

OpenCode 순정

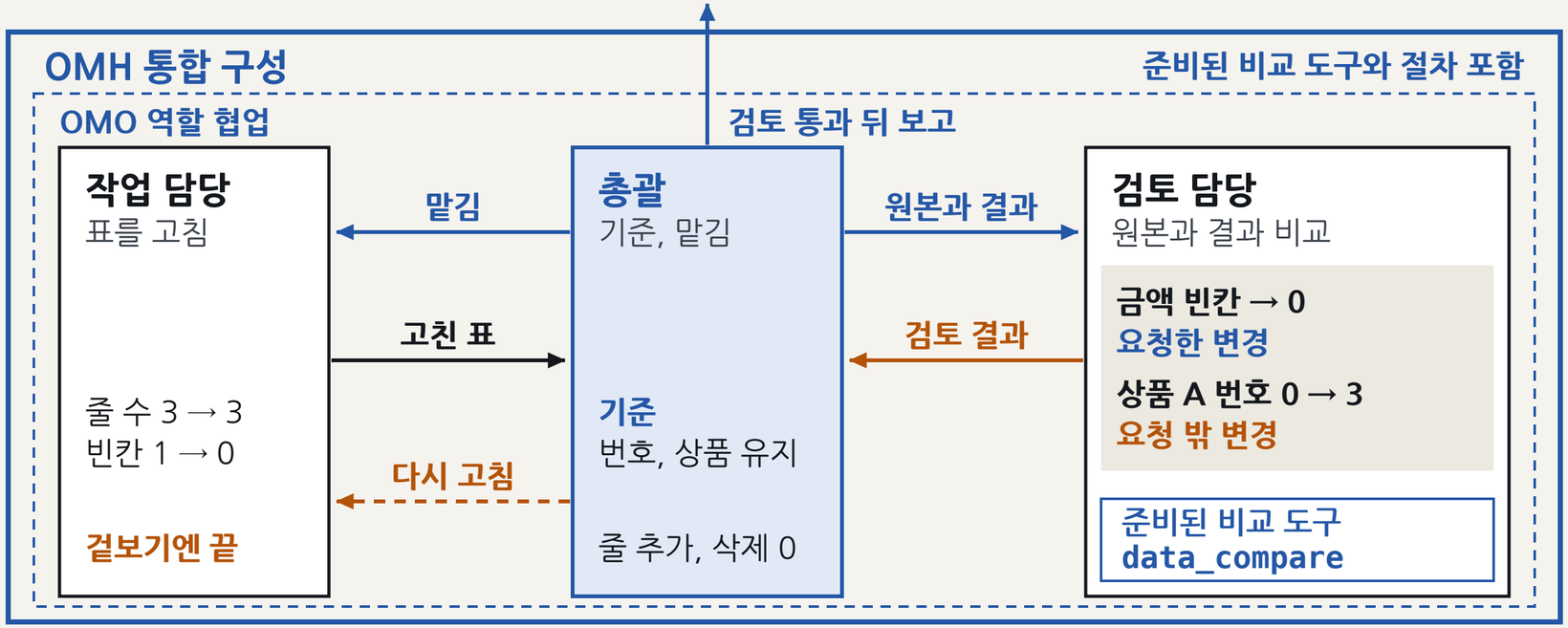
같은 일을 직접 꾸려 진행

1 **표를 고침**
금액 빈칸에 0을 채움

2 **원본과 비교**
비교 코드를 짜서 맞춰 봄

3 **기준에 따라 검토**
번호, 상품이 그대로인지

4 **필요하면 다시 맡김**
고칠 것을 되돌림



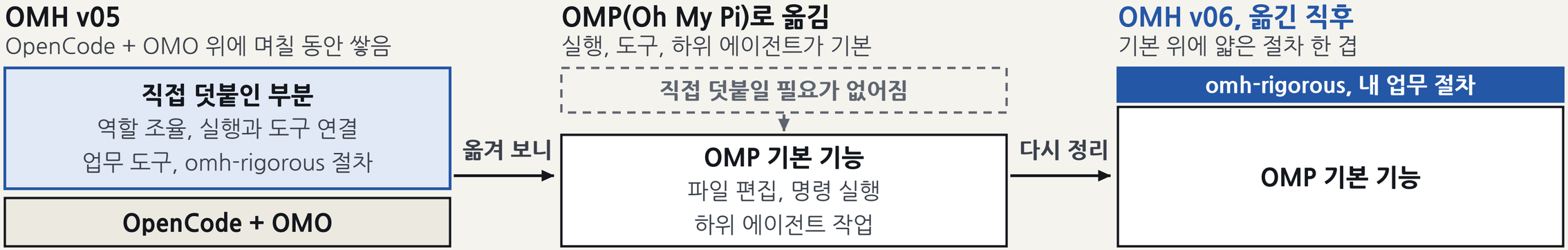
쓰는 이유 **반복할수록, 복잡할수록 커지는 하네스의 효용**

분업, 비교, 재검토 절차를 재사용해 복잡한 반복 작업의 준비와 확인 부담을 줄이기

사례 2 이어서 OMH에서 OMP로

질문 직접 덧붙인 하네스, 바탕이 바뀌면 얼마나 남을까?

며칠을 만들었는데, 옮겨 보니 이미 있었다



그 뒤, 지금: 필요한 스킬을 더해 키워 가는 중

필요에 따라 골라 씀

`omh-rigorous`	`omh-refactor`	`cross-network-summary`	`omh-knowledge`	`kar-plain`
조사와 검증	**구조 정리**	**다른 망으로 넘길**	**OKF로 지식 정리**	**쉬운 말로**
근거와 대조	리팩터링	맥락 요약	다음 작업에 재사용	풀어 설명

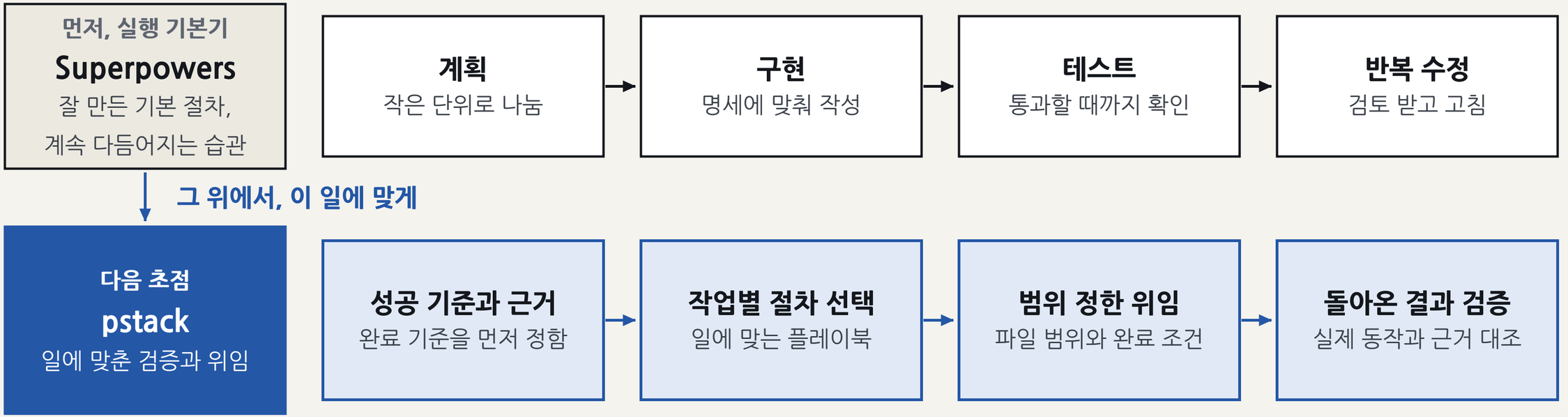
작성자 경험 **며칠 동안 v05까지 만들었는데, 상당 부분은 이미 기본 기능이었음**

v06에는 필요한 절차만 얇게. 옮기고 하나만 남았고, 그 뒤 하나씩 더해 발전시키는 중

참고 A Superpowers와 pstack

질문 계획하고 테스트하는 습관이 기본이 되면, 하네스는 무엇을 더 챙겨야 할까?

기본기는 모델에, 업무 기준과 위임은 하네스에



겹치는 부분 **두 묶음 모두 계획, 위임, 검토, 테스트를 갖춤**

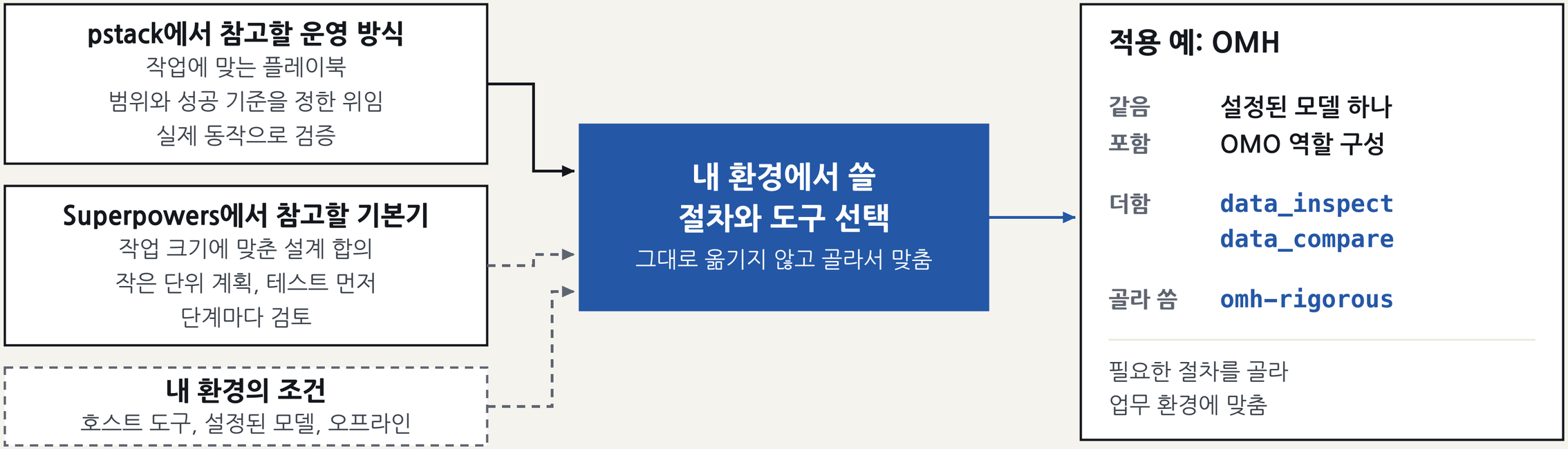
계획하고 시험하는 기본기는 모델에 스며들고, 하네스는 이 일의 기준과 근거, 위임을 챙김

출처: pstack 0.15.29 (cursor/plugins 111cf6a), Superpowers 7.0.0 (obra/superpowers b739a8a), 2026-10-21 기준 편집 요약

참고 B pstack과 Superpowers

질문 다른 팀의 스킬 묶음에서 무엇을 가져와야 할까?

가져올 것은 기능 목록보다 작업 방식



정리 OMH는 모델을 유지하고, 절차와 확인 도구를 업무에 맞게

출처: pstack 0.15.29 (cursor/plugins 111cf6a), Superpowers 7.0.0 (obra/superpowers b739a8a), 2026-10-21 기준 편집 요약

플러그인

장면

쓰기 지침, 서식 기능, 계정 연결 설정을 한 묶음으로 설치.

계정 연결까지 끝난 걸까?

여러 기능, 지침, 설정을 한 번에 설치하고 버전으로 관리하려고 배포하는 패키지 단위. 범위와 설치 방식은 제품마다 다름



헷갈리기 쉬운 점 플러그인 안에 MCP 서버 설정이 들어갈 수는 있어도, 배포 단위와 통신 규약은 다른 층

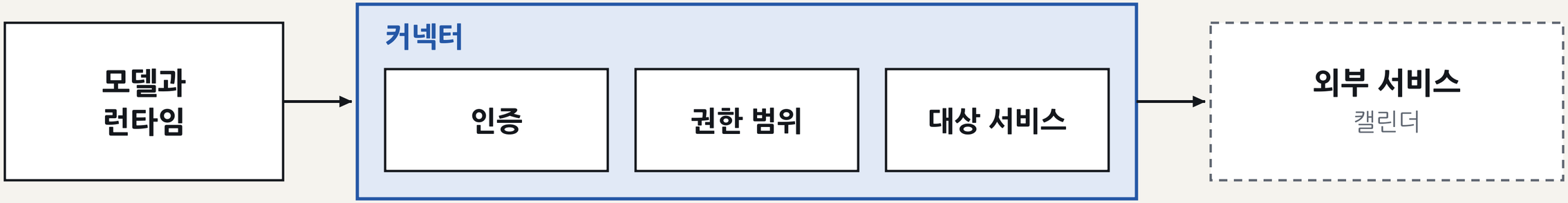
커넥터

장면

이번 주 회의 일정을 캘린더에서 가져오기.

AI가 캘린더를 바로 읽을 수 있을까?

모델과 런타임이 외부 서비스, 데이터 저장소, 다른 앱의 기능에 닿도록 만든 연결 지점. 연결 설정과 인증 절차까지 포함



✓ 연결과 권한이 살아 있으면
일정 목록, 권한 범위만큼

✕ 연결이 없거나 만료되면
연결 오류

목록에 보이는 것과
실제로 호출되는 것은 별개
확인은 호출 결과로

헷갈리기 쉬운 점 커넥터는 외부와의 연결 지점, 도구는 모델이 호출하는 기능. 커넥터 하나가 도구 여럿을 제공하기도 함

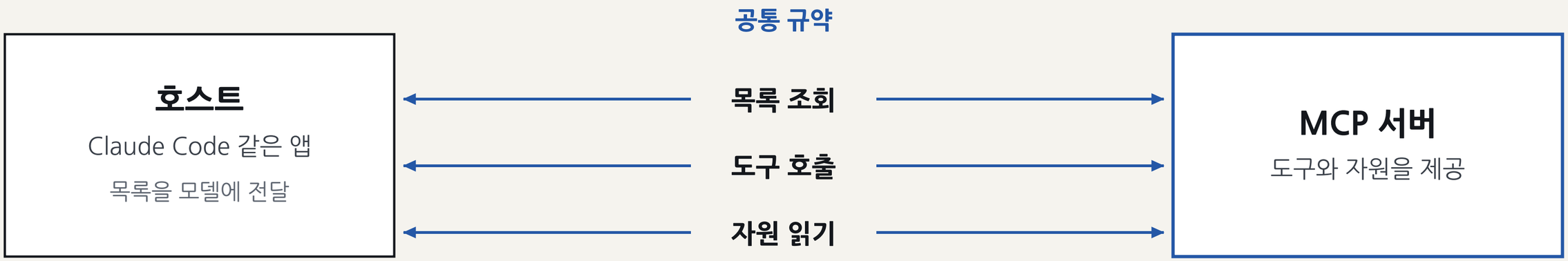
MCP

장면

여러 도구가 파일 목록과 내용을 같은 방식으로 주고받아야 하는 상황.

도구마다 따로 약속을 만들어야 할까?

Model Context Protocol. 호스트 앱과 외부 도구, 자원 서버가 도구 목록 조회, 도구 호출, 자원 읽기를 주고받는 방식을 정한 통신 규약.



✓ **규약을 따르는 것**
메시지 형식과 교환 절차가 맞음

다른 문제

✕ **접근 권한을 얻는 것**
계정 연결과 허용 범위는 따로 정함

헷갈리기 쉬운 점 로컬 함수나 셸 명령은 MCP 없이도 도구. 모든 도구가 MCP인 것은 아님

사례 3 / 5 Grokbot

질문 하네스를 쓰려면 늘 켜 둔 내 컴퓨터가 꼭 있어야 할까?

내 맥미니 없이도, 클라우드에서 도는 봇 넷



어디서 돌아가나

직접 운영
클식이
장비와 실행 환경을 우리가 관리

클라우드 서비스
Grok Bot
서비스가 늘 켜진 컴퓨터를 제공

늘 켜 둔 내 장비 없이도 하네스를 쓸 수 있음

서비스마다 권한과 이용 조건은 다름
다른 서비스는 다음 쪽, 참고 C

본문으로, 23쪽

질문 직접 만든 봇이 없어도, 비슷한 걸 써 볼 수 있을까?

맥미니 없이도, 클라우드에서 쓰는 개인 에이전트

dots — OpenAI

들어가는 곳
ChatGPT에서 시작

맡기는 일
연결한 앱으로 이어서 일하기

사람이 보는 곳
결과 검토와 판단

Grok Bot — xAI

들어가는 곳
봇별 대화, 그룹 대화

맡기는 일
늘 켜진 컴퓨터로 앱 작업

사람이 보는 곳
필요할 때 승인 요청

Muse — Meta

들어가는 곳
앱이나 WhatsApp

맡기는 일
메일, 여행, 양식 처리

사람이 보는 곳
메일 발송, 구매 전 확인

셋의 공통점

- 클라우드의 자기 컴퓨터
- 연결한 앱
- 이어지는 기억
- 사람의 확인

출처: 각 회사 공식 발표와 도움말 (2026-10-21 확인). 이용 조건과 제공 지역은 서비스마다 다름

3장

AI는 무엇을 알고 있을까?

03

이번 호출에 무엇이 들어갔을까? 지시와 입력, 처리 단위와 한계, 저장과 불러오기, 다시 찾아 쓰기. AI교양으로 이어지는 맥락

프롬프트 — 지시와 제약

컨텍스트 — 이번 호출의 입력

토큰 — 세는 단위

컨텍스트 창 — 한 호출의 한계

메모리 — 저장과 불러오기

LLM wiki — 정리해 다시 찾기

사례 AI교양 — 작가의 확정

프롬프트

장면

"회의 준비해 줘" 한 줄, 그리고 안건, 참석자, 지난 결정을 넣은 요청.

어느 쪽이 결과를 더 좌우할까?

모델 입력 중 지시, 질문, 역할, 제약을 담은 부분. 일회성 요청일 수도, 여러 실행에 되풀이해 쓰는 지속 지침일 수도

요청 A

"회의 준비해 줘"

↓

일반적인 준비 목록

빈 곳은 모델의 추측으로

요청 B

"회의 준비해 줘"
+ 안건 세 줄
+ 참석자 명단
+ 지난 회의 결정

↓

안건 중심 준비안

입력에 없는 사실은 여전히 확인 필요

헷갈리기 쉬운 점 프롬프트는 지시 문장, 컨텍스트는 실제로 입력된 전체. 폴더 속 규칙 파일은 읽혀야 비로소 입력

컨텍스트

장면

지난 회의록은 경로만 알려 주고 내용은 넣지 않은 상태.

AI는 지난 결정을 떠올릴 수 있을까?

특정 호출에서 모델이 실제로 입력받은 모든 내용. 지시, 파일 내용, 도구 결과, 이전 대화 중 포함된 부분까지

이번 호출에 들어간 입력

- 지시: "지난 결정 반영해서 회의 준비해 줘"
- 규칙 파일 내용
- tickets.csv 앞 20행
- 도구 결과
- 이전 대화 중 포함된 부분

디스크에만 있는 것

- 지난 회의록 내용
- `checklist.md`
- tickets.csv 21행부터

읽어서 넣어야 근거로 쓸 수 있음

헷갈리기 쉬운 점 저장된 파일과 지금 읽은 파일은 별개. 메모리는 나중에 불러올 저장 정보, 컨텍스트는 지금 입력에 든 정보

토큰

장면

같은 안내 메일, 한국어판과 영어판.

글자 수가 같으면 토큰 수도 같을까?

모델이 입력과 출력을 처리할 때 세는 텍스트 조각의 단위. 단어 수, 글자 수와는 다른 기준

한국어	안내	메일	을	보내	드립니다
영어	We	'll	send	the	notice

위 분할은 설명용 가상 예시

글자 수가 같아도 토큰 수는 다를 수 있다

언어별 보편 비율은 없음
정확한 수는 해당 모델의
계산 방식으로 확인

헷갈리기 쉬운 점 토큰은 측정 단위, 컨텍스트 창은 그 단위로 잰 한계. 여기서는 개수를 어림하지 않음

컨텍스트 창

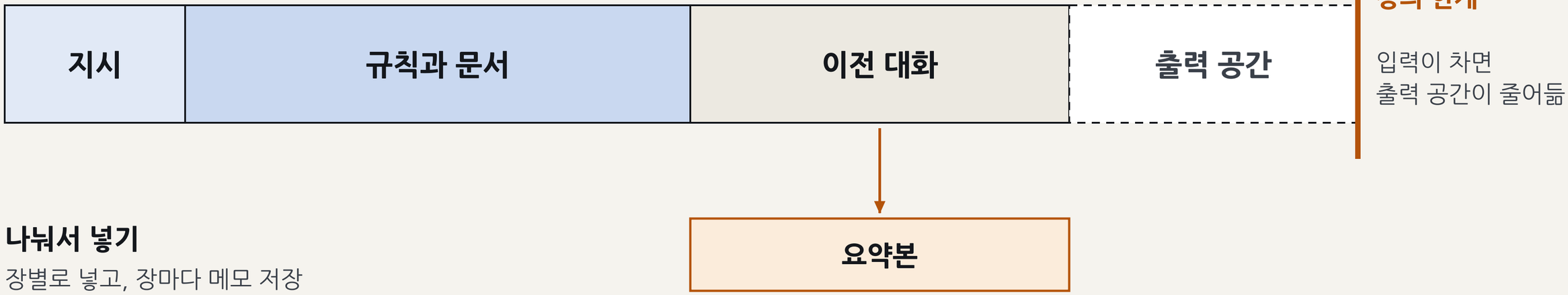
장면

100쪽 계약 문서를 한 번에 넣고 검토 요청.

앞부분 내용도 끝까지 기억될까?

모델이 한 번의 호출에서 입력과 출력을 함께 처리할 수 있는 토큰 범위의 한계. 크기는 모델과 제품 설정에 따라 다름

한 번의 호출



창의 한계

입력이 차면
출력 공간이 줄어듦

나눠서 넣기

장별로 넣고, 장마다 메모 저장

압축: 이전 대화를 요약해 넣는 분량을 줄임. 세부 일부는 사라질 수 있음

헷갈리기 쉬운 점 압축은 창에 넣는 내용의 분량을 줄이는 일, 창의 크기는 그대로. 메모리는 호출 밖에 저장했다가 다시 불러오는 일

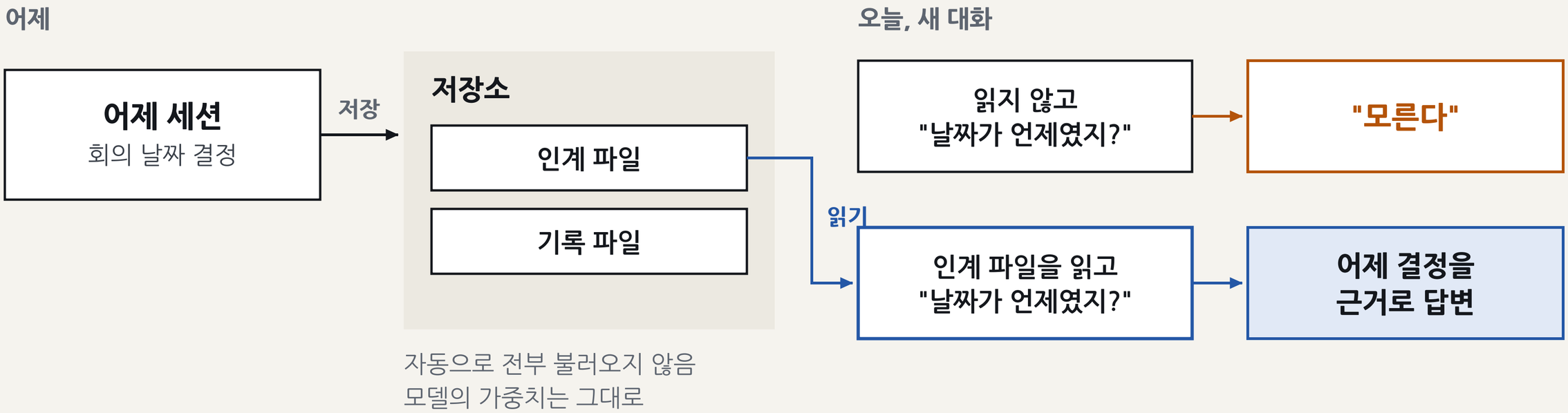
메모리

장면

어제 정한 회의 날짜는 인계 파일에.

오늘 새 대화를 열면 그 결정이 따라올까?

세션이나 실행이 끝난 뒤에도 다시 불러올 수 있게 파일이나 저장소에 남긴 정보. 모델의 가중치는 그대로



헷갈리기 쉬운 점 세션이 이어진다고 메모리가 생기는 것은 아님. 재학습도 아니고, 달라지는 건 입력에 들어가는 내용뿐

LLM wiki

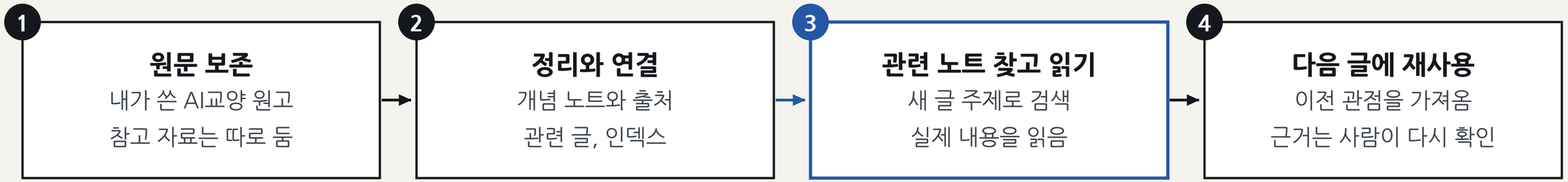
장면

지난 AI교양 글에서 정리한 개념을 새 글에도 쓰고 싶다.

저장해 둔 노트, 새 세션이 저절로 떠올릴까?

LLM과 함께 지식을 정리하고 연결해, 필요할 때 다시 찾아 쓰는 방식

쌓아 둔 기록을, 다음 작업의 재료로



파일로 남긴 위키 (예: Obsidian)

이번 작업의 컨텍스트로 읽어 옴

LLM wiki는 정리하고 쓰는 방식, OKF(Open Knowledge Format)는 출처와 상태를 적는 형식

저장만으로 기억되는 건 아님. 필요한 노트를 찾아 읽는 단계까지.

사례 4 / 5 AI교양

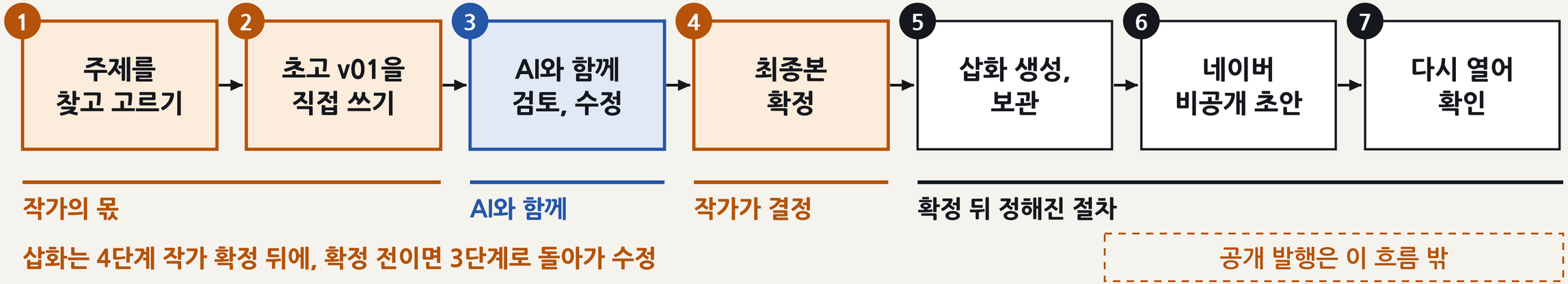
질문 최종본 확정 전인데 삽화부터 만들자는 요청. 만들어도 될까?

흐름을 받치는 장치까지 보면, 업무용 하네스

규칙
초고 v01 보존과 버전, 확정 문장 불변

컨텍스트
해당 글, 수정 이력, 관련 근거

도구
파일 저장, 이미지 생성, 초안 작성



상태
현재 버전, 확정 여부, 단계별 결과, 초안 번호

검증
저장본과 비교, 초안 다시 열어 확인

완료 기록을 설명용으로 인용 (이번 작업으로 제작하지 않음). 별도 상태 머신이 전환을 강제한다는 뜻은 아님

쉬는 시간

돌아오면 4장

누가 이어서 일할까?

지금까지: 모델과 실행, 일하는 방법을 담는 곳, 모델이 아는 것

4장

누가 이어서 일할까?

04

대화를 묶는 단위, 실제로 일하는 작업자, 맡겨진 하위 실행. 이 셋을 가른 뒤,
상황에 맞춰 페르소나를 고르는 사례로.

세션
작업 맥락

워커
실행 담당

서브에이전트
맡겨진 하위 실행

사례 CEO Agent
상황별 선택

세션

하나의 작업 흐름을 묶어 두는 대화와 상태의 단위. 입력, 출력, 도구 기록, 진행 상태를 한데 묶음

장면

같은 회의 준비 대화를 노트북과 휴대폰에서 함께 열기.

같은 대화를 두 화면에서 보면 작업자도 두 명일까?



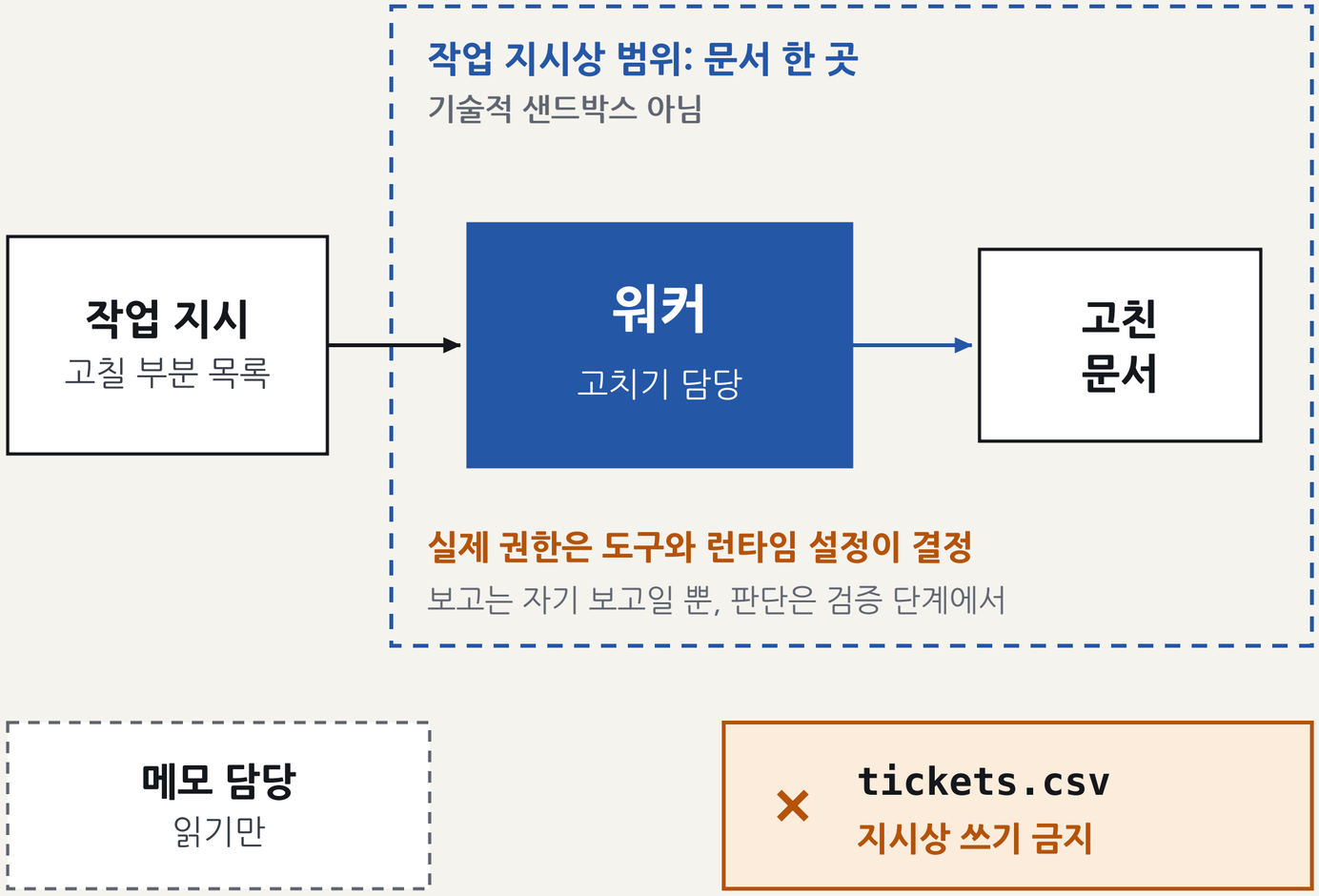
화면 두 개, 세션 하나

파일을 고치는 작업자는 하나

헷갈리기 쉬운 점 세션 자체가 작업을 실행하지는 않음. 저장된 이력 가운데 이번 호출에 들어간 부분만 컨텍스트

워커

배정받은 작업을 실제로 수행하는 작업자.
읽기 전용 조사인지, 파일 수정인지에
따라 권한과 작업 범위도 달라짐



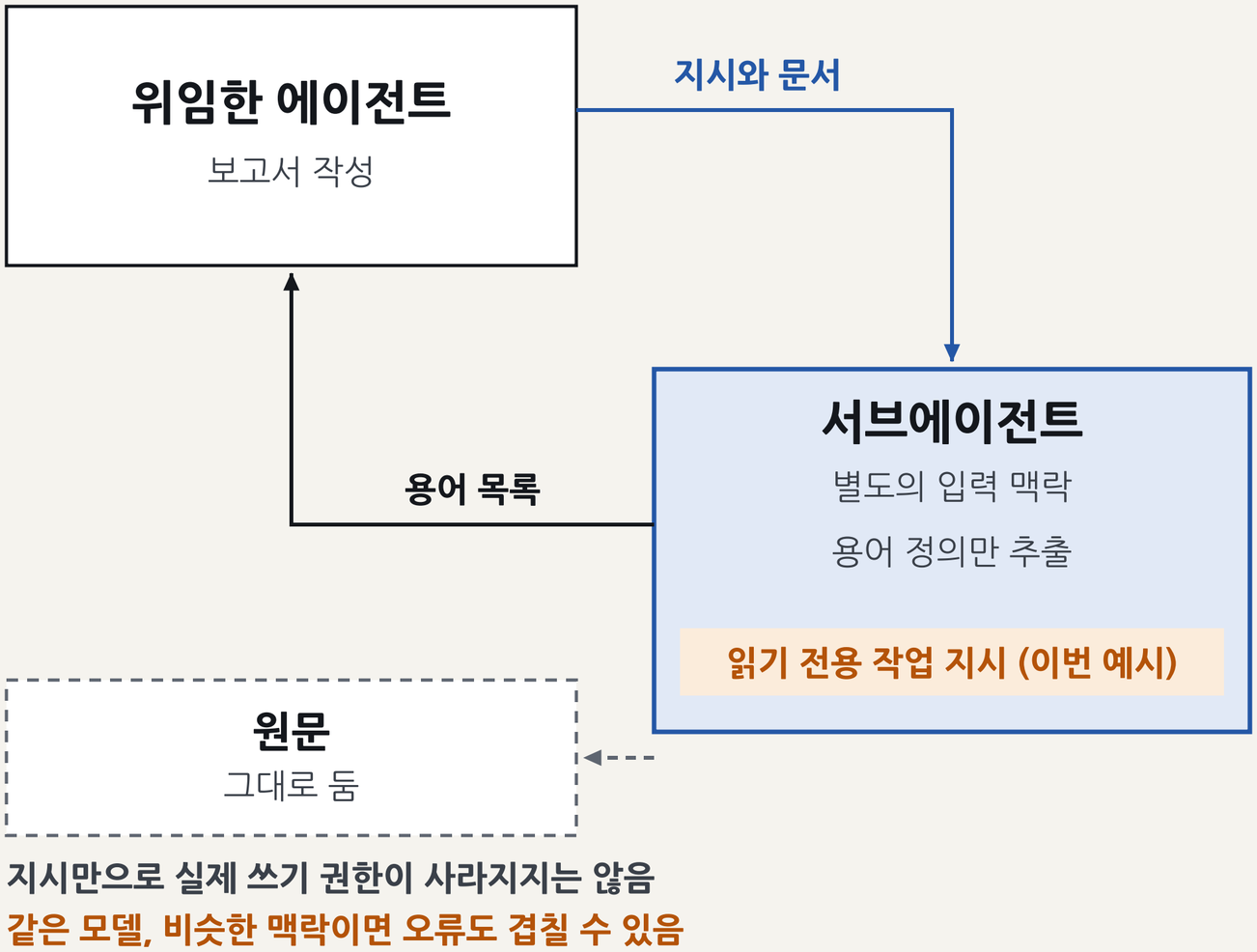
장면

한 사람은 고치기만, 다른 사람은 메모만.

문서를 실제로 고칠 담당은 누구로 정할까?

헷갈리기 쉬운 점 워커는 맡은 일을 하는 역할, 서브에이전트는 상위와 하위의 위임 관계. 하위 에이전트가 워커 역할을 맡기도 함

서브에이전트



다른 에이전트가 맡긴 작업을 별도의 입력 맥락에서 수행하고 결과를 돌려주는 하위 에이전트. 권한은 설정에 따라 읽기 전용일 수도, 쓰기 가능일 수도

장면

긴 문서에서 용어 정의만 뽑는 일을 떼어 맡기기.

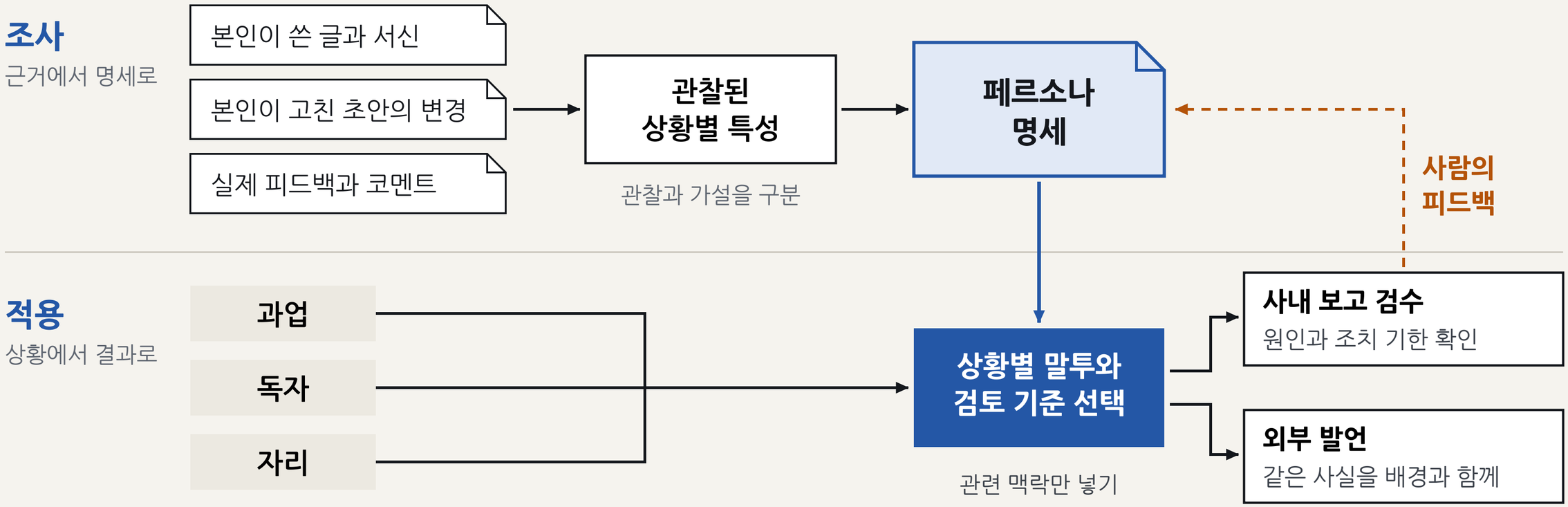
그 작업자가 원문을 고쳐도 될까?

헷갈리기 쉬운 점 오케스트레이션은 배정 방식, 서브에이전트는 실행 단위. 서브에이전트를 둔다고 독립 검토가 되지는 않음

사례 5 / 5 CEO Agent

질문 같은 사실이라면 사내 보고와 외부 발언의 강조도 같아야 할까?

조사로 만든 페르소나, 상황에 맞게 골라 쓰기



같은 사실(가상): 분기 일정 지연 두 건. 원자료와 인물 정보는 비공개. 모델 학습은 없고, 각 단계도 독립된 작업자가 아님

5장

끝났다는 말을 어떻게 믿을까?

05

작업 흐름과 시작 조건, 완료 주장과 확인 근거. 실행 위치와 사람의 개입까지,
다섯 사례를 나란히



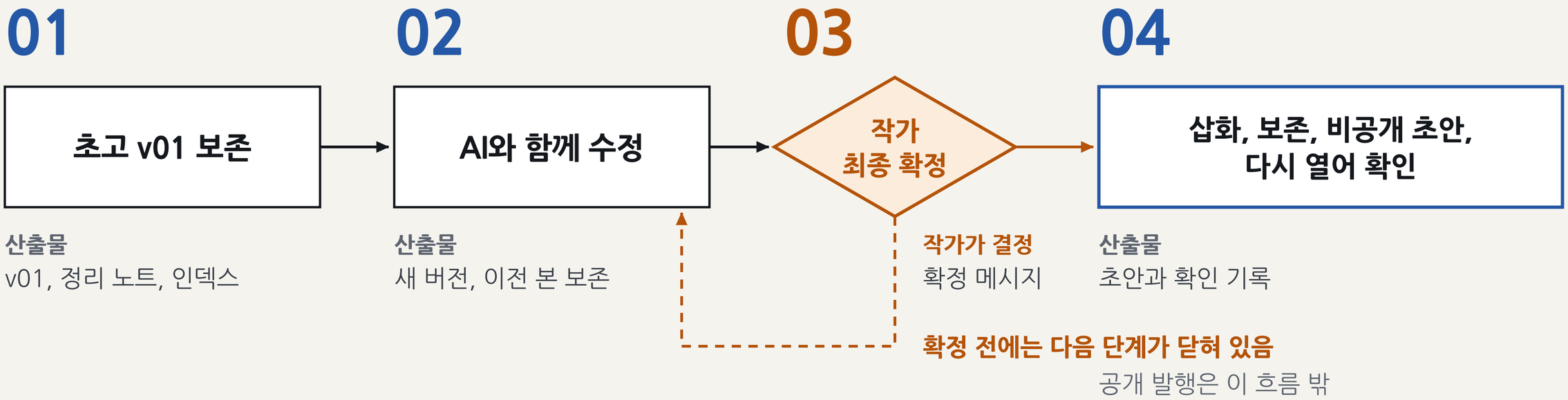
워크플로우

장면

작가의 초고 v01 도착. 최종본은 아직 확정 전.

삽화부터 만들어도 될까?

입력, 단계, 단계별 산출물, 다음 단계로 넘어가는 조건을 정해 둔 작업 흐름. 사람이 손으로 따라가도 워크플로우



헷갈리기 쉬운 점 자동화는 그중 일부를 조건이 맞을 때 시작하는 장치, 하네스는 이 단계를 돌리고 검증하는 틀

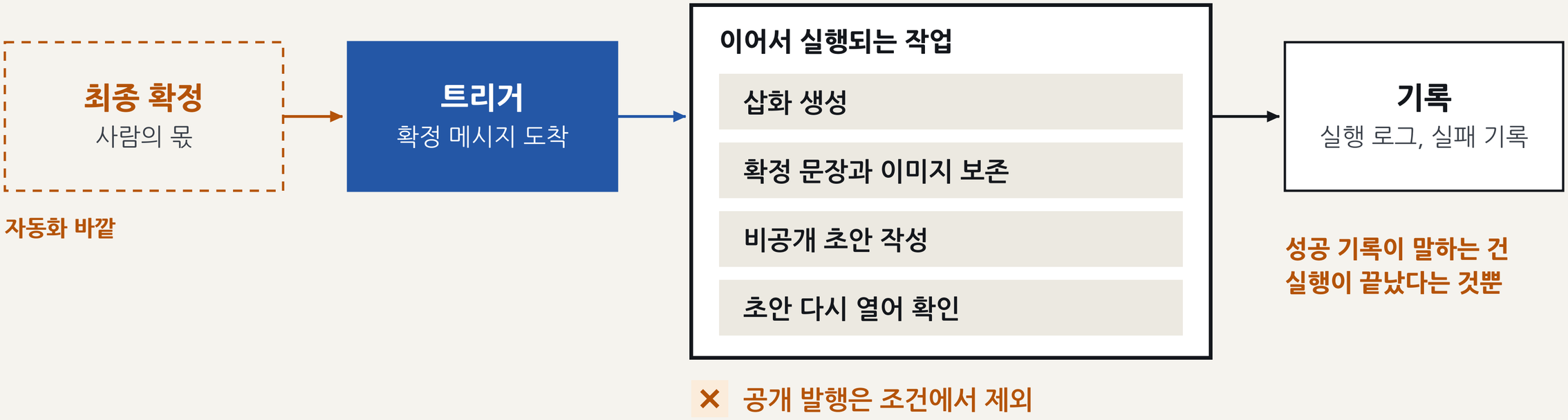
자동화

시간, 이벤트, 앞 단계 결과 같은 조건이 충족되면 사람이 다시 시작하지 않아도 정해진 작업을 실행하게 만든 장치

장면

최종 확정 메시지 도착. 저장과 초안 작성까지 이어서 돌리고 싶은 상황.

확정 메시지 하나로 뒤의 제작 전체를 시작해도 될까?



자동화 바깥

성공 기록이 말하는 건 실행이 끝났다는 것뿐

✕ **공개 발행은 조건에서 제외**

헷갈리기 쉬운 점 자동 기억은 저장과 불러오기의 문제, 자동화는 실행 시작의 문제. 이 장면은 설명용이며 실제로 돌리지 않음

완료 보고

장면

작업자의 보고는 "검토 완료" 한 줄.

이 한 줄로 검토가 끝났다고 할 수 있을까?

작업자가 "끝났다"고 남긴 기록. 한 일과 확인했다고 주장하는 내용을 담을 뿐, 그 자체가 검증 증거는 아님

완료 보고	주장
상태	완료
산출물	`report.md`
확인 항목	건수 검증 완료
미확인	없음

대조 결과	확인
✓ 파일 존재	예
✗ 건수 일치	아니오
보고서 42, 원본 행 40, 고유 티켓 38	

보고는 완료, 검증은 실패. 둘을 나란히 기록

헷갈리기 쉬운 점 실행 도구의 done은 실행이 끝났다는 증거일 수는 있어도, 요구 조건을 만족했다는 증거는 아님

검증

장면

주간 요약에 적힌 "이번 주 문의 42건".

누가 어떻게 확인할까? 한 번 보면 충분할까?

미리 정한 기준으로 산출물, 숫자, 인용, 파일 상태, 실행 결과를 작업자의 말과 독립된 방법으로 확인해 판단하는 과정

항목	확인 방법	결과 기록
건수 일치 원본 데이터와 대조	원본 행 수와 고유 ticket_id 수를 명령으로 세어 요약과 대조	**원본 40, 고유 38, 보고서 42** **실패**
판단이 어려운 행 목록 원본 데이터와 대조	빈 칸이나 형식이 다른 날짜가 있는 행 번호를 뽑아 보고서와 대조	**빠진 행이 있으면 실패로 기록**
근거 인용 사람의 확인	주요 주장마다 근거 행 번호가 인용됐는지 읽고 확인	아직 확인 전

기준은 작업 전에. 한 항목 통과가 전체 통과는 아님

헷갈리기 쉬운 점 같은 작업자가 다시 읽는 것은 약한 점검. 리뷰는 판단, 검증은 기준에 따른 확인

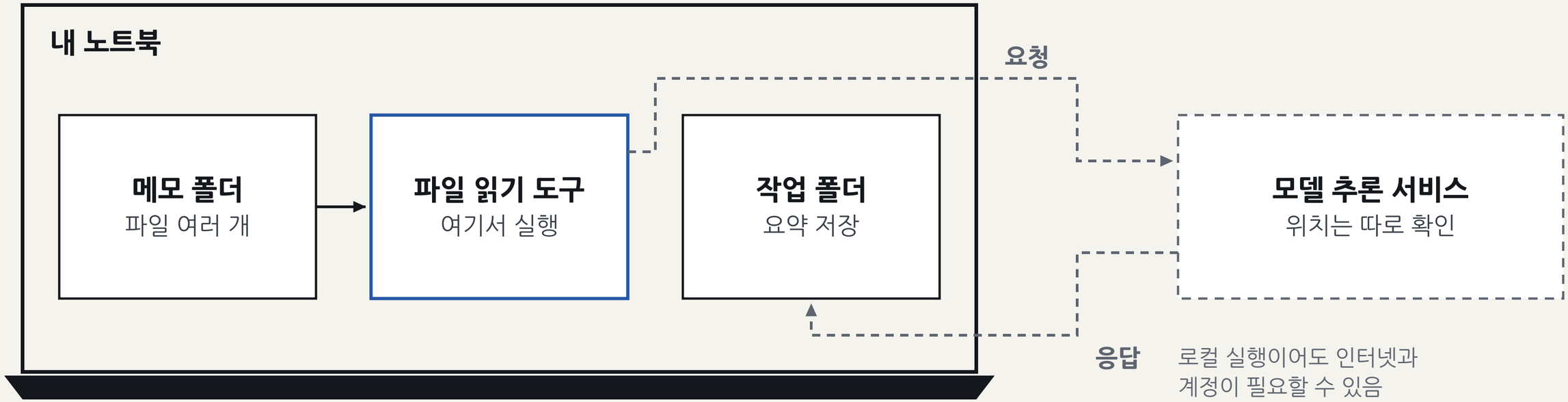
로컬 실행

장면

AI 도구가 노트북의 메모 폴더를 읽고 요약.

요약 답변도 노트북에서 만들어질까?

도구와 프로세스가 사용자의 컴퓨터에서 돌고, 파일 접근과 명령이 그 기기 안에서 일어나는 방식. 모델 추론 위치는 따로 따짐



헷갈리기 쉬운 점 "로컬에서 돌렸다"가 모델도 기기 안에서 돌았다는 뜻은 아님. 로컬이 곧 안전하다는 뜻도 아님

클라우드 실행

도구, 워커, 모델 추론 중 일부나 전부가 사용자 기기가 아닌 외부 서버에서 도는 방식. 어느 구성 요소가 그런지 함께 밝혀야 함

장면

초안은 외부 서버의 모델이 쓰고, 파일은 노트북에 저장.

"클라우드 작업"이라고 한 단어로 불러도 될까?

구성 요소		내 기기	외부 서버
모델 추론	초안 텍스트 생성		여기
초안 파일 저장	노트북에 기록	여기	
파일 도구	읽고 쓰는 기능	여기	
메일 발송	이 장면에서는 안 함	실행하지 않음	

"클라우드 작업" 한 마디로는 어느 부분인지 알 수 없음

헷갈리기 쉬운 점 클라우드 실행과 리모트 컨트롤은 다른 축. 공유 범위는 제품 설정에 따르며, 모든 파일이 어디서나 공유된다고 가정하지 않음

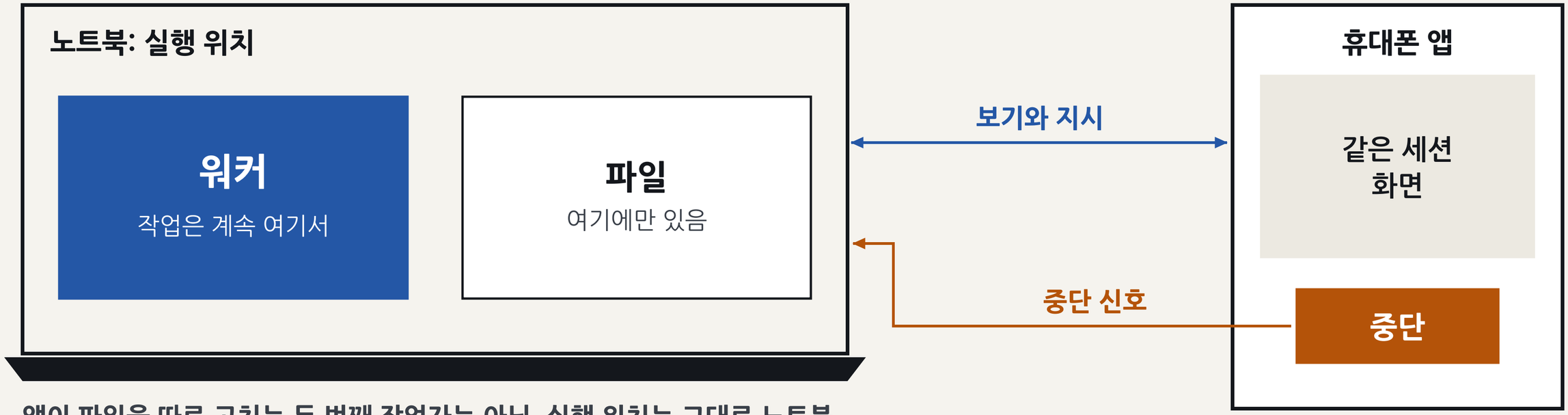
리모트 컨트롤

장면

노트북에서 돌아가는 작업, 휴대폰으로 보다가 멈추고 싶을 때.

멈춤을 누르면 작업 장소가 휴대폰으로 옮겨질까?

실제 실행은 원래 환경에서 계속하고, 다른 화면이나 기기에서 그 세션을 보고 지시하거나 중단 신호를 보내는 개입 방식



앱이 파일을 따로 고치는 두 번째 작업자는 아님. 실행 위치는 그대로 노트북

헷갈리기 쉬운 점 실행 위치와 개입 방식은 다른 축. 한 경로로 연결됐다고 다른 경로의 앱에서도 보인다는 보장은 없음

종합

다섯 사례, 같은 요소, 다른 무게

규칙 | 컨텍스트 | 도구와 실행 위치 | 상태 | 검증과 사람의 최종 책임

AI교양	**흐름과 넘어가는 조건, 작가의 확정**	다시 열어 확인
클식이	**실행 위치와 개입 방식**	워커와 작업 확인 화면
OMH	**제약에 맞춘 도구와 권한, 증거와 한계**	사내 모델 품질 미검증
Grokbot	**클라우드 서비스에서 도는 봇 넷**	작성자가 확인한 구성
CEO Agent	**조사한 페르소나를 상황에 맞게 고르는 흐름**	단계는 독립 작업자가 아님

측정한 성과의 비교가 아니라, 설계와 기록 구조의 비교

질의응답

질문과 이야기

질문이 없다면: 가장 헷갈렸던 용어 하나와 그 이유는?

1장

5 모델
6 에이전트
7 런타임
8 하네스
9 **클식이 ■**

2장

11 오케스트레이션
12 도구
13 스킬
14 **OMH ■**
18 플러그인
19 커넥터
20 MCP
21 **Grokbot ■**

3장

24 프롬프트
25 컨텍스트
26 토큰
27 컨텍스트 창
28 메모리
29 LLM wiki
30 **AI교양 ■**

4장

33 세션
34 워커
35 서브에이전트
36 **CEO Agent ■**

5장

38 워크플로우
39 자동화
40 완료 보고
41 검증
42 로컬 실행
43 클라우드 실행
44 리모트 컨트롤

숫자는 슬라이드 번호, 사각형 표시는 사례. 확인되지 않은 내용은 확인이 필요하다고 답하기

만들어 보고도, 남아 있는 질문

공부할수록, 오히려 확신이 줄었어요.
에이전트가 더 똑똑해지면 해결될 일은 아닐까.
우리가 더해야 할 구조는 어디까지일까.

누군가는 필요성을 느끼고,
누군가는 저처럼 질문이 더 생겼을지도 모르겠어요.

그래도 직접 시도하고 알아간 시간은 남겠죠.
오늘은 답 하나보다, 각자의 질문 하나가 남았으면 해요.